

Documentation EKF sensor fusion

Armando Palermo

December 2025

1 Modello del Sistema

Il modello del robot è rappresentato dallo stato:

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}, \quad \mathbf{u} = \begin{bmatrix} v \\ \omega \end{bmatrix}, \quad \boxed{\begin{array}{l} x_{k+1} = x_k + v \cos \theta_k \Delta t, \\ y_{k+1} = y_k + v \sin \theta_k \Delta t, \\ \theta_{k+1} = \theta_k + \omega \Delta t \end{array}}$$

Nel modello si utilizza come velocità lineare il valore proveniente dal topic delle ruote in corrispondenza di `twist.linear.x`, poiché la componente `twist.linear.y` presenta valori molto piccoli (ordine di 10^{-2}) e viene interpretata come rumore. Inoltre, dato che la struttura del Warthog è assimilabile a un sistema *skid-drive*, si è assunta una rappresentazione del controllo tramite due sole velocità: una lineare v e una angolare ω .

I Jacobiani utilizzati nella linearizzazione EKF sono:

$$F_x = \begin{bmatrix} 1 & 0 & -v \sin \theta \Delta t \\ 0 & 1 & v \cos \theta \Delta t \\ 0 & 0 & 1 \end{bmatrix}, \quad F_u = \begin{bmatrix} \cos \theta \Delta t & 0 \\ \sin \theta \Delta t & 0 \\ 0 & \Delta t \end{bmatrix}.$$

2 Matrice Q del Rumore di Controllo

La rumorosità dei comandi è modellata tramite:

$$Q = \begin{bmatrix} \sigma_v^2 & 0 \\ 0 & \sigma_\omega^2 \end{bmatrix}, \quad \begin{array}{l} \sigma_v^2 = \alpha_1 v^2 + \alpha_2 \omega^2, \\ \sigma_\omega^2 = \alpha_3 v^2 + \alpha_4 \omega^2. \end{array}$$

Inizialmente la matrice Q era estratta dal topic delle ruote fornito nella rosbag. La matrice in questione però era statica e per generalizzare meglio si è deciso di utilizzare una matrice di incertezza dipendente dal controllo stesso.

3 Flusso della Fusione

L'algoritmo di fusione elabora i messaggi dei diversi sensori disposti in un'unica timeline ordinata per timestamp. Al primo messaggio disponibile viene eseguita l'inizializzazione dello stato, ricavando posizione e orientamento dal sensore odometrico, convertendo il quaternion in angolo di yaw e impostando la covarianza iniziale. Contestualmente viene memorizzato il tempo di riferimento ed inizializzato il vettore di controllo $u = [v, \omega]^T$, così da evitare predizioni con passo temporale nullo.

Per ogni nuovo messaggio in arrivo il filtro verifica se il sensore associato consente l'aggiornamento dell'ingresso di controllo; in tal caso vengono estratte le velocità e aggiornato u . Se più sensori producono misure allo stesso istante, l'ingresso viene comunque aggiornato ma senza ripetere la predizione, che avviene solo quando il tempo effettivamente avanza. Quando il timestamp corrente è maggiore del precedente si calcola Δt e si esegue la fase di predizione applicando il modello di movimento $f(x, u, \Delta t)$, quindi si aggiornano stato e covarianza mediante i Jacobiani F_x e F_u e la matrice di rumore Q dipendente dai comandi.

Ogni volta che un sensore fornisce una misura compatibile con la fase di correzione, il filtro effettua l'update dell'EKF indipendentemente dal fatto che sia avvenuta o meno una predizione nello stesso ciclo. Per tenere conto di sensori con rate differenti, la matrice di misura R viene scalata dinamicamente in funzione della frequenza con cui lo stesso topic compare consecutivamente. La correzione procede calcolando l'innovazione $y = z - Hx$, normalizzando l'angolo nel caso sia presente, e applicando il guadagno di Kalman per aggiornare stato e covarianza. A ogni iterazione vengono infine salvate la posa stimata e la covarianza associata, costruendo così l'intera traiettoria stimata dal filtro sulla timeline.

4 Struttura del programma

Le classi sensore forniscono all'EKF le funzioni necessarie per interrogare ciò che il sensore effettivamente offre. Per definire lo scopo operativo di un sensore vengono utilizzati, in particolare, i seguenti metodi:

- `has_correction_phase()`: indica se il sensore fornisce misure per la correzione.
- `has_update_u_phase()`: indica se il sensore aggiorna il comando $\mathbf{u}$ (predizione).
- `get_z_measurements(row)`: estrae il vettore misura $\mathbf{z}$ dal messaggio.
- `get_H_matrix()`: restituisce la matrice di osservazione H .
- `get_R_matrix(row)`: restituisce la covarianza R della misura.
- `get_u_parameter(row)`: estrae il controllo $\mathbf{u} = [v, \omega]$, quando disponibile.

4.1 Gestione della matrice R del sensore DLIO

L'analisi della rosbag mostra che il sensore DLIO pubblica una matrice di covarianza della posa quasi costante per tutti i campioni, con varianze molto piccole e poco realistiche per un odometro. Poiché tali valori non sembrano riflettere un'incertezza realmente stimata dall'algoritmo, l'uso diretto di questa matrice avrebbe attribuito al DLIO un peso eccessivo nella fusione, penalizzando sensori come il GPS.

Per correggere questo squilibrio è stato applicato uno scaling alla matrice R_{dllo} , aumentando la varianza delle componenti di posizione e, in misura minore, di yaw. Inoltre l'EKF modula dinamicamente il peso del DLIO tramite il parametro α , che dipende dalla frequenza con cui il topic compare consecutivamente nella timeline: misure molto frequenti ricevono automaticamente un peso minore, mentre misure più rare risultano più influenti.

Questa doppia strategia produce una matrice R_{dllo} più coerente e garantisce che il contributo del DLIO sia bilanciato rispetto agli altri sensori.